```
Running migrations:
 Rendering model states... DONE
 Unapplying promoters.0006_promoter_birth_squashed_0007_promoter_death... OK
```

The `--plan` option to `migrate` shows what would happen if you run `migrate` without any arguments. As you've just undone the squashed 0006 script, using `--plan` tells you that a migration is due:

```
(venv) RiffMates$ python manage.py migrate --plan
Planned operations:
promoters.0006_promoter_birth_squashed_0007_promoter_death
    Add field birth to promoter
    Add field death to promoter
```

Not only does `--plan` tell you which scripts will be executed, but it even includes the details about their effects. In this case, that is the addition of the birth and death fields.

Scary options to migrate

There are a few more arguments to `migrate` that allow you to muck around with the database. Recall from chapter 4 that Django keeps the current state of migration in a table called `django_migrations`. The `--fake` and `--fake-initial` options update this table as if a migration had run, without actually running it. The `--fake-initial` option restricts the change to only the initial migration. The `--run-syncdb` option creates tables for the apps but without running migrations. You use these three options if you have existing database structures you're trying to Django-ify or you're trying to bypass the migration system for performance reasons.

Using any of these options can result in the contents of the `django_migrations` table being out of sync with the migration files in your project. Be careful, as a mismatch may result in Django not detecting future changes or rerunning a migration that has already been done.

The `--prune` option removes any migrations from the `django_migrations` table that no longer exist. A few extra rows in the database are not going to hurt anything, so this really is a neat-freak kind of choice.

The `makemigrations` command also has some useful optional arguments. You can use `--dry-run` to see what the result of calling `makemigrations` would be, without it actually creating the migration script.

One challenge you may encounter when working with multiple developers is two people making a change to a model on different branches in the code repo at the same time. Whoever merges last gets a merge conflict. The `Model` state needs to be fixed by addressing the two overlapping changes. Additionally, you'll have two migration files with the same starting number, one from your change and one from your colleague's.